

PilotFish CSV XML Guide – Deep Dive (26R1.11)

Derived from decompiled eip.war.hs.26R1.11 + Sandbox demos

August 4, 2026

At a glance (plain English)

CSV XML is PilotFish’s XML view of flat CSV files. Comma- (or pipe-/caret-) separated text becomes nested XML you can map with XSLT, then optionally turn back into CSV. It is not Excel XML, not RFC 4180-as-XML, and not one universal schema — **two dialects** appear depending on which module converts the file.

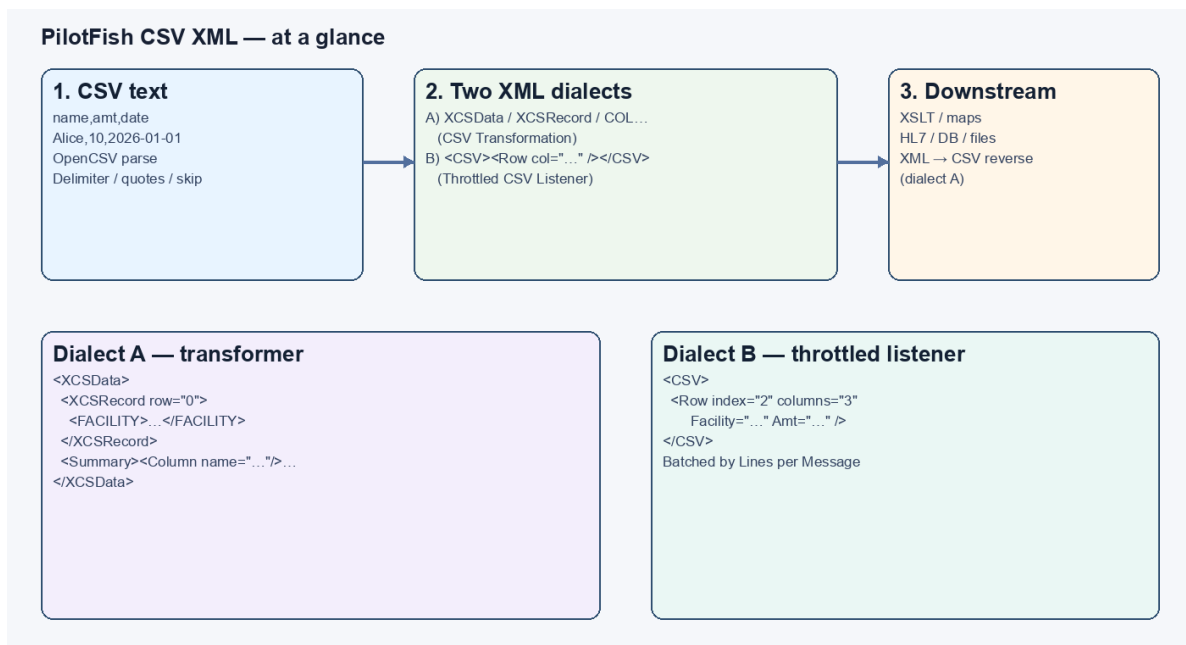


Figure 1: At a glance (plain English)

- **Dialect A (CSV Transformation):** <XCSDData> / <XCSRecord> / column element tags + optional <Summary>.
- **Dialect B (Throttled CSV File Listener):** <CSV> / <Row ... colName="value" /> attribute rows, usually **batched**.
- OpenCSV parses the text; delimiter, quotes, skip-lines, and column naming are module options.
- Only Dialect A has a built-in **XML -> CSV** reverse path.

Purpose

This guide explains how **PilotFish** represents **CSV** as **XML** in **eip.war.hs.26R1.11**: both dialects, configuration that changes the tree, XSLT patterns, reverse transform rules, and traps. Companion to:

- PilotFish-Throttled-CSV-File-Listener-Reference-26R1.11.md
- CSV Transformation processor (CSVTransformationProcessor / CSVTransformationModule)

Provenance	Value
WAR	eip.war.hs.26R1.11
CSV to/from XML	com.pilotfish.eip.modules.transform.CSVTra
Processor wrapper	com.pilotfish.eip.modules.transform.CSVTra
Listener batch XML	com.pilotfish.eip.modules.file.throttling.
Fork (raw CSV, not XML)	com.pilotfish.eip.modules.csv.CSVForkingMo
Parser library	OpenCSV (CSVParser / CSVReader / CSVWriter)

Derived from CFR decompile plus Sandbox interface 88363 Log to HL7 DFT
(formats/CSV to XML/).

1. Mental model

CSV bytes

```
|
+-- CSV Transformation (direction CSV to XML)
|   -> Dialect A: XCSDData / XCSRecord / <COL>...</COL> / Summary
|   -> (later) XSLT, processors, transports
|   -> optional XML to CSV (same module)
|
+-- Throttled CSV File Listener (Convert to XML = true)
|   -> Dialect B: <CSV><Row attrs.../></CSV> per batch
|   -> synchronous route per batch
|
+-- CSV Forking (raw text batches; does NOT emit CSV XML)
```

Choose deliberately. Maps written for //XCSRecord/PatientName will not see Throttled Listener Row/@PatientName without an XSLT rewrite (and the reverse).

2. Dialect A — CSV Transformation (XCSDData)

Produced by CSVTransformationModule.executeTransformDataToXML.

2.1 Skeleton

```
<XCSDData>
  <XCSRecord row="0">
    <FACILITY>COCLSU</FACILITY>
    <LOCATION>W.CT</LOCATION>
    <PATIENTNAME>LOCKARD,DEBORAH LYNN</PATIENTNAME>
    <!-- one child element per column -->
  </XCSRecord>
  <XCSRecord row="1">...</XCSRecord>
  <Summary>
    <Column name="FACILITY">
      <MaxLength>...</MaxLength>
      <NumNulls>...</NumNulls>
      <AllInteger>>false</AllInteger>
      <AllNumeric>>false</AllNumeric>
      <MaxValue>...</MaxValue>
      <MinValue>...</MinValue>
      <TotValue>...</TotValue>
    </Column>
    <!-- one Column per named field (+ Unknown_Data if used) -->
  </Summary>
</XCSDData>
```

Piece	Meaning
XCSDData	Root wrapper (always written)
XCSRecord/@row	Zero-based data row index after skips
Column child tags	Header-derived or configured names (see §2.2)
Summary	Per-column profiling trailer for this conversion

2.2 How column tag names are chosen

Configuration	Behavior
Columns empty	First CSV line is consumed as header . Each cell is <code>trim().toUpperCase()</code> then <code>XMLUtil.toValidXMLName</code> . Blank header -> Field1, Field2, ...
Columns set	Comma-separated list becomes tag names (each run through <code>toValidXMLName</code>). First line is not treated as header unless you also Lines to Skip .
Extra cells beyond name count	Tag <code>Unknown_Data</code> (same name reused for every overflow cell)

Configuration	Behavior
Columns to Suppress	Matching names emit empty contribution / can omit with Suppress Empty Tags

Sandbox pattern (named columns + skip header line):

```
Columns = Facility, Location, PatientName, ...
Lines to Skip = 1
Suppress Empty Tags = true
Delimiter = ^
Ignore Quotes = true
```

2.3 Configuration that shapes Dialect A

UI (approx.)	Tag	Default	Direction tab	Effect
Delimiter	Delimiter ,		Basic	First char after Java unescape (\t etc.)
Auto-detect Delimiter	DetectDelimiter	false	To XML	First non-alphanumeric (excluding -_) on first non-empty line; overrides Delimiter
Columns	Columns	empty	To XML	Explicit names (see above)
Lines to Skip	LinesToSkip	1	To XML	OpenCSV withSkipLines before read
Columns to Suppress	ColumnsToSuppress		To XML	Comma list of tag names to suppress
Suppress Empty Tags	SuppressEmptyTags	false	To XML	Omit empty cell elements
Ignore Quotes	IgnoreQuotes	false	To XML	OpenCSV withIgnoreQuotations
Force UTF-8	UseUTF8Tag	false	To XML	UTF-8 reader/writer
Wrap Output in Quotes	OutputQuotes	true	From XML	CSVWriter quote char " vs \0

UI (approx.)	Tag	Default	Direction	tab	Effect
Output Column Headers	<code>OutputColumnHeaders</code>	<code>xmlHeaders</code>	From XML		Emit header line from first <code>XCSRecord</code> child names

Parser fixed choices on to-XML: quote `"`, escape `\`, `strictQuotes=false`, leading WS not ignored.

2.4 XML -> CSV (Dialect A only)

`executeTransformDataFromXML` SAX-parses the stream:

1. On each end of `XCSRecord`, write one CSV line from collected child text values (document order).
2. If **Output Column Headers**, first record's **child element local names** become the header line once.
3. **Summary and XCSDData are not emitted as rows**; only nested fields under each `XCSRecord` matter.
4. Empty `XCSRecord` still writes a line (possibly empty cells).

Author maps/XSLT so each `XCSRecord`'s children are **only** CSV columns, in the order you want on the wire.

2.5 Escaping

To-XML escapes `<`, `>`, `&` in cell text (`xmlEscape`). Quotes may be trimmed via `trimQuotes` when building values. Listener Dialect B uses `StringEscapeUtils.escapeXml11` on **attribute** values instead.

3. Dialect B — Throttled CSV Listener (CSV / Row)

When **Convert to XML** is true, each batch is:

```
<CSV>
  <Row index="2" columns="3" Facility="COCLSU" Location="W.CT" PatientName="..." />
  <Row index="3" columns="3" Facility="..." Location="..." PatientName="..." />
</CSV>
```

Attribute	Meaning
<code>index</code>	1-based file line index (includes header/footer region counting)
<code>columns</code>	Parsed cell count for that line
<i>column labels</i>	From listener Columns list (<code>c1</code> default); only <code>i < columns.size()</code> cells get attributes

Related listener options

Option	Role for XML shape
Lines per Message	How many data rows per <CSV> batch
Delimiter / Expecting Quotes	OpenCSV separator; <code>ignoreQuotations = !ExpectingQuotes</code>
Columns	Attribute names (required list for conversion)
Header Option	Skip headers, or Repeat them as leading <Row> in every batch
Skip/Repeat	
# of Header / Footer Lines	Lines excluded from “data” region
Validate Contents + Column Patterns	Pre-scan whole file: column count + regex per column (\\, escape in pattern list)

Batches also get transaction attributes (`BatchNumber`, `LineCount`, `IsLastBatch`, ... — see listener reference).

CSV Forking operates on **raw CSV text** batches; it is not a third CSV XML dialect.

4. XSLT patterns

4.1 Dialect A (Sandbox HL7 DFT path)

```
//XCSRecord[string-length(PatientName) > 0]
<xsl:for-each select="//XCSRecord[string-length(PatientName) > 0]">
  <!-- build HL7 / target XML from Facility, SpecimenNumber, ... -->
</xsl:for-each>
```

Skip EOF / trailer junk rows (demo keeps sentinel rows where `Facility` holds markers) with predicates on required fields.

Group/dedupe demos often key on `XCSRecord/@row` or a business field:

```
XCSRecord/PatientAcctNum
```

Ignore or strip <Summary> when feeding XML->CSV or secondary maps — leave it out of forked “one record” documents unless needed for profiling.

4.2 Dialect B

```
/CSV/Row/@Facility
//Row[@columns='3']
```

Attribute-centric maps; convert to element form in XSLT if you need element-based routes:

```
<xsl:for-each select="/CSV/Row">
  <XCSRecord row="{position()-1}">
    <Facility><xsl:value-of select="@Facility"/></Facility>
    <!-- ... -->
```

```
</XCSRecord>
</xsl:for-each>
```

5. Worked example (Dialect A from Sandbox)

Interface: 88363 Log to HL7 DFT -> format CSV to XML

1. Source CSV uses ^ delimiter; first line headers skipped (LinesToSkip=1).
2. Explicit Columns=Facility,Location,PatientName,...
3. Transformer emits XCSDData / XCSRecord (see formats/CSV to XML/transform-test.xml).
4. Format XSLT / route XSLTs select non-empty PatientName rows and build DFT / reports.
5. Optional later stages fork or regroup by facility codes.

Abbreviated:

```
<XCSDData>
  <XCSRecord row="0">
    <Facility>COCLSU</Facility>
    <Location>W.CT</Location>
    <PatientName>LOCKARD,DEBORAH LYNN</PatientName>
    <SpecimenNumber>SU23:LSU:2666</SpecimenNumber>
    <!-- ... -->
  </XCSRecord>
  <Summary>
    <Column name="Facility">...</Column>
  </Summary>
</XCSDData>
```

(Tag case in live output follows configured names / toValidXMLName — keep XPath aligned with actual tags.)

6. Quirks and traps

1. **Two dialects** — Listener <CSV>/<Row> is not what CSV Transformation writes; do not mix maps blindly.
2. **Header double-handling** — With Columns filled in Dialect A, set **Lines to Skip** (usually 1) or the header becomes a data XCSRecord.
3. **Unknown_Data collision** — Extra cells share one tag name; document order only distinguishes them.
4. **Summary always appended** — Dialect A always writes <Summary> after rows; reverse transform ignores it, but copy/XPath /* consumers may be surprised.
5. **Summary type flags** — In this WAR's ColumnStats, AllInteger / AllNumeric initialize to Java false and are only cleared on parse failure, so they typically stay false even for numeric columns. Prefer MaxLength / NumNulls / sums when profiling.
6. **Unused numeric extremes** — MaxValue/MinValue stay at Double.MIN_VALUE / MAX_VALUE when no numeric parse succeeded (visible in Sandbox test XML).

7. **Quote polarity differs** — Listener **Expecting Quotes** false means ignore quotations **true**. Transformer exposes **Ignore Quotes** directly.
8. **Auto-detect delimiter** — First “special” character on a line; hyphen/underscore/space never chosen; fails closed to ,.
9. **Throttled ConvertToXML + text path** — Decompiled `processFileStream` runs the plain-text batch loop **after** the XML block without an early return. Treat Convert-to-XML routes carefully; validate that only one payload style reaches the route (see listener quirks).
10. **XML->CSV needs XCSRecord** — Nested trees that omit that element name produce no CSV lines.
11. **CSV Forking is not CSV XML** — Forks raw lines (optional header prepend); use Transformation if you need Dialect A.

7. Operational recommendations

Goal	Guidance
Standard inbound CSV -> map	CSV Transformation Dialect A + explicit Columns + LinesToSkip
Huge file, limited concurrency	Throttled CSV Listener; Convert to XML only if maps need Dialect B
Round-trip CSV	Same Dialect A module; Output Column Headers + Wrap Quotes to match consumers
HL7 / enrichment maps	Filter empty/sentinel <code>XCSRecords</code> before message build
Strict shapes	Listener Validate Contents + regex Column Patterns or validate in XSLT after Dialect A
Ordered batches	Concurrent Transactions = 1 on throttled listener

8. Source index

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
CSV to/from XML module	<code>com/pilotfish/eip/modules/transform/CSVTransformation</code>
Processor wrapper	<code>.../transform/CSVTransformationProcessor.java</code>
Listener Dialect B	<code>.../file/throttling/CSVThrottledDirectoryListener.j</code>
CSV forking (raw)	<code>com/pilotfish/eip/modules/csv/CSVForkingModule.jav</code>

Sandbox reference:

```
eip-root/interfaces/88363 Log to HL7 DFT/formats/CSV to XML/
  format.xml
  transform-test.xml
  transform.xslt
```

9. See also

- Documents/Listeners/26R1.11/PilotFish-Throttled-CSV-File-Listener-Reference-26R1.11.md
- Documents/Listeners/26R1.11/PilotFish-SQL-XML-Guide-26R1.11.md
- Documents/Processors/26R1.11/PilotFish-EDI-XML-Guide-26R1.11.md

Guide derived from WAR decompile and Sandbox demos – CSV XML – 26R1.11 – August 2026.